

YieldLens: Stability-Aware Sensor Reduction for Semiconductor Yield Monitoring Using Chronological Drift Validation

PROJECT REPORT

Submitted by

KARTHICK S

2116231801079

in partial fulfilment for the award of the degree of

AD23A38 - MANUFACTURING ANALYTICS

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Under the guidance of

Mrs. K. Selvarani M.E., (Ph.D)

Assistant Professor (SS), Department of Artificial Intelligence and Data Science



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

RAJALAKSHMI ENGINEERING COLLEGE

Rajalakshmi Nagar, Thandalam, Chennai – 602 105, Tamil Nadu, India

OCT - 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report **“YieldLens: Stability-Aware Sensor Reduction for Semiconductor Yield Monitoring Using Chronological Drift Validation”** is the Bonafide work of **“Karthick S (231801079)”**, who carried out this project work for the course AD23A38 – Manufacturing Analytics under my supervision.

SIGNATURE

Mrs. K. Selvarani M.E., (Ph.D).

SUPERVISOR

Assistant Professor (SS) Department of
Artificial Intelligence & Data Science
Rajalakshmi Engineering College
Rajalakshmi Nagar Thandalam, Chennai
- 602 105

SIGNATURE

Dr. J M Gnanasekar M.E., Ph.D.

HEAD OF THE DEPARTMENT

Professor and Head
Department of Artificial Intelligence &
Data Science Rajalakshmi Engineering
College Rajalakshmi Nagar Thandalam,
Chennai - 602 105

Submitted to mini-project viva voce examination held on:

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially I thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. I sincerely thank our respected Chairman **Mr. S. MEGANATHAN, B. E, F.I.E.**, our Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, and our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution. I sincerely thank **Dr. S. N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete my work in time. I express my sincere thanks to **Dr. J M GNANASEKAR, Ph.D.**, Professor and Head of the Department of Artificial Intelligence and Data Science for his guidance and encouragement throughout the project work. I convey my sincere and deepest gratitude to our internal guide, **Mrs. K. Selvarani M.E., (Ph.D).**, Assistant Professor (SS), Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College for her valuable guidance throughout the course of the project. I am very glad to convey our sincere gratitude to our Project Coordinator, **Mrs. K. Selvarani M.E., (Ph.D).**, Assistant Professor (SS), Department Artificial Intelligence and Data Science for her useful tips during our review to build our project.

Karthick S

ABSTRACT

Modern semiconductor fabrication lines collect readings from hundreds of sensors per wafer, most of which are redundant, noisy, or unrelated to the actual failure mechanism. Standard feature selection methods evaluate sensor importance on a single static data split, which ignores a known property of fabrication processes: sensor behavior drifts over time due to mechanical wear, chemical variation, and calibration change. A sensor that appears predictive today may not remain predictive months later. This paper presents YieldLens, a method for selecting a small, monitoring-worthy subset of sensors by measuring their predictive stability across chronologically ordered time batches, rather than their importance on one snapshot. Using the public SECOM dataset (590 sensors, $\sim 1,567$ wafers), we reduce the sensor set to 17 sensors and show that models trained on this reduced set outperform models trained on the full 590-sensor set, across two independent model families (XGBoost and Random Forest), on AUC, Precision@K, and Recall@K. We validate the stability metric itself using a synthetic drift-injection experiment, and provide sensor-level explainability using SHAP and interpretable risk binning. We report all method limitations directly, including small sample size and single-dataset scope.

Keywords: Feature selection, stability selection, concept drift, semiconductor manufacturing, yield prediction, class imbalance, SHAP, virtual metrology

TABLE OF CONTENTS

Section	Title	Page
	Bonafide Certificate	
	Acknowledgement	
	Abstract	
1.	Introduction	7
	1.1 Overview of the Problem Domain	7
	1.2 Motivation for This Project	7
	1.3 Scope of This Report	8
2.	Problem Statement	9
	2.1 Problem Description	9
	2.2 Key Challenges	9
	2.3 Research Gap	9
	2.4 Research Objective	10
3.	Dataset Description & Preprocessing	11
	3.1 Dataset Description	11
	3.2 Data Loading	11
	3.3 Preprocessing Method Selection	11
	3.4 Description of Preprocessing Steps	12
	3.5 Outcome of Preprocessing	13
4.	Model Development & Methodology	14
	1.1 Model Selection	14
	4.2 Model Justification	14
	4.3 Model Description	14
	4.3.1 Stability Selection	14
	4.3.2 Tripwire Audit	15
	4.3.3 Shortlist Construction	15
	4.3.4 Drift-Injection Validation	15
	4.3.5 Classification Models	15
	4.4 System Architecture	15
	4.5 Methodology - Step-by-Step Pipeline	16
5.	Metrics to Measure	17
	5.1 KPIs Used - Definitions & Description	17
	5.2 Reason for Selection	17
6.	Result Review	19

	6.1 Output Description	19
	6.1.1 Sensor Reduction Outcome	19
	6.1.2 Drift-Injection Validation Outcome	20
	6.1.3 Model Performance Outcome	20
	6.1.4 Explainability Outcome	21
	Conclusion	22
	References	24

LIST OF TABLES

Table	Title
Table 3.1	Chronological Batch Summary — Rows, Failures, and Fail Rate per Batch
Table 6.1	Top Sensors by Stability Score and Average LASSO Importance
Table 6.2	Drift Injection Validation — Stability Score Before vs. After Noise Injection
Table 6.3	Model Performance — Full (590) vs. Reduced (~17 Sensors)
Table 6.4	Top 5 Sensors by Mean Absolute SHAP Value

LIST OF FIGURES

Figure	Title
Figure 3.1	Process Drift — Fail Rate Across Chronological Batches
Figure 4.1	YieldLens System Architecture
Figure 6.1	Sensor Stability vs. Importance Scatter Plot, All Sensors vs. Shortlisted (~17)
Figure 6.2	Model Comparison — Full (590) vs. Reduced (~17 Sensors)
Figure 6.3	Top Sensors by SHAP Importance — Reduced Model, Mean SHAP Value
Figure 6.4	Risk Bins for sensor_59

1. INTRODUCTION

1.1 Overview of the Problem Domain

Semiconductor fabrication is one of the most data-intensive manufacturing environments in modern industry. A single wafer moving through a fab pass under hundreds of in-line sensors that continuously log process parameters - temperature, pressure, flow rates, plasma characteristics, and dozens of other electro-mechanical readings - at every stage of production. The SECOM dataset used in this project is a real-world example of this: **590 sensor readings** are captured per wafer, alongside a pass/fail outcome and a timestamp indicating when that wafer was processed.

The underlying business problem is yield monitoring: fabs want to catch a defective wafer *before* it moves further down the line, because the cost of rework or scrap increases sharply the later a defect is detected. In principle, more sensors should mean better defect prediction. In practice, monitoring, storing, and modeling hundreds of correlated, noisy, and often redundant sensor channels is expensive and operationally unwieldy. Engineers on the shop floor cannot act on a 590-dimensional SHAP explanation - they need a short, trustworthy list of sensors that genuinely drive failure risk.

This is compounded by two further realities of semiconductor data:

- **Severe class imbalance:** failures are rare events. In this dataset the overall fail rate is only about 6.6%, meaning a naive model can achieve high accuracy while never actually catching a defect.
- **Process drift:** the manufacturing process itself changes over time (tool wear, recipe changes, calibration drift, raw material batch changes), so a sensor that was predictive in January may not be predictive in June. Most published feature-selection studies on SECOM ignore this and pick features from a single, static snapshot of the data.

1.2 Motivation for This Project

YieldLens was built to address the gap between *"which sensors look important today"* and *"which sensors stay meaningfully predictive over time."* Rather than treating feature selection as a one-off statistical exercise, this project treats it as a **temporal reliability problem** - sensors are only useful for long-term monitoring if they remain informative across changing process conditions.

The project therefore combines four ideas into one pipeline: (1) chronological, leakage-free data splitting, (2) LASSO-based feature selection repeated across time-ordered batches to produce a stability score per sensor, (3) a safety-net "tripwire audit" that rescues rare-but-critical sensors that stability selection alone would discard, and (4) a synthetic drift-injection test that verifies the stability score actually reacts to drift rather than being a static artifact.

1.3 Scope of This Report

This report documents the full YieldLens pipeline end-to-end: the SECOM dataset and how it was cleaned and batched; the sensor-reduction methodology (LASSO stability selection + tripwire audit + drift validation); the two classification models trained on the resulting 17-sensor shortlist versus the full 590-sensor baseline (XGBoost and Random Forest); the evaluation metrics chosen for an imbalanced, capacity-constrained inspection setting (AUC, Precision@K, Recall@K); and the resulting model performance, explainability (SHAP + OptBinning risk bins), and the FastAPI service built to serve predictions and explanations.

2. PROBLEM STATEMENT

2.1 Problem Description

Existing feature-selection approaches for semiconductor yield prediction rank sensors using a **single snapshot** of the data, implicitly assuming that a sensor's importance stays constant over time. In a real fab, this assumption breaks down: process drift changes sensor behavior, so a feature-selection method built on one static split may recommend a shortlist of sensors that quietly stops being predictive weeks later. This risk is amplified by the highly imbalanced nature of yield data, where actual failure samples are rare and easy to overlook during naive ranking.

2.2 Key Challenges

- Identifying a small set of sensors that remains predictive **across time**, not just within one train/test split.
- Avoiding the discarding of **rare but operationally critical** sensors - ones that may appear unimportant on average but "activate" specifically during failure events.
- Selecting features under class imbalance, where standard feature-importance rankings can be dominated by noise from the majority (pass) class.
- Validating, rather than assuming, that a stability-based selection method is actually sensitive to drift.

2.3 Research Gap

As summarized in the literature review conducted for this project:

Prior Work	Approach	Limitation	YieldLens Response
McCann & Johnston (2008)	Baseline SECOM feature selection	Static, single split	Chronological batching applied to all 590 sensors
Guyon & Elisseeff (2003)	Feature selection survey	No temporal analysis	Drift-aware embedded feature selection
Tibshirani (1996)	LASSO feature selection	No stability over time	LASSO applied across time batches
Meinshausen & Bühlmann (2010)	Stability selection	Random resampling only	Stability measured across chronological batches
Gama et al. (2014)	Concept drift survey	Not applied to sensor selection	Drift concept applied directly to sensor stability
He & Garcia (2009)	Imbalanced learning	General-purpose methods	SMOTE + class-weighted learning, applied specifically to yield prediction

2.4 Research Objective

To design a sensor-reduction method that selects sensors based on their **predictive stability across chronologically ordered time batches**, rather than importance on a single snapshot - producing a smaller, drift-aware sensor shortlist that is *at least as reliable* as the full sensor set for downstream failure prediction.

3. DATASET DESCRIPTION & PREPROCESSING

3.1 Dataset Description

The project uses the **SECOM dataset** (UCI Machine Learning Repository), a widely used benchmark for semiconductor manufacturing process monitoring.

- **Total records:** 1,567 wafers
- **Raw features:** 590 anonymized sensor readings, plus a timestamp and a pass/fail label
- **Label encoding:** the raw dataset encodes -1 as Pass and 1 as Fail; this was remapped in preprocessing to 0 = Pass, 1 = Fail for standard binary-classification convention
- **Class balance:** approximately 93:7 (Pass:Fail), i.e. an overall fail rate of ~6.6%, confirming the dataset is significantly imbalanced
- **Temporal structure:** every record carries a timestamp, which is central to this project's methodology - the data is sorted chronologically rather than treated as an i.i.d. sample

3.2 Data Loading

The raw sensor readings (secom.data) and labels (secom_labels.data) are loaded separately and merged on row order. Column names are auto-generated as sensor_0 ... sensor_589. The label column is cleaned into a binary 0/1 format, and - critically - the merged dataframe is **sorted by timestamp** before any further processing, so that all downstream splits respect real chronological order rather than the original file order.

3.3 Preprocessing Method Selection

Three preprocessing concerns were addressed, each chosen specifically because of properties of this dataset:

1. **Missing values:** SECOM sensor readings contain missing entries at varying rates per sensor. Per-sensor **median imputation** was selected over mean imputation because sensor readings can be skewed or contain outlier spikes; the median is more robust to this than the mean, and it avoids dropping any of the 590 original sensors before the reduction step is allowed to run on the full feature space.
2. **Class imbalance:** because failures are rare (~6.6%), imbalance is handled in two complementary ways: **class-weighted LASSO** during feature selection (so rare failure batches still influence which sensors are selected) and **SMOTE (Synthetic Minority Over-sampling)** during model training, applied strictly to the training data only to avoid leaking synthetic information into the test set.
3. **Chronological batching instead of random shuffling:** rather than a random train/test split (the norm in most SECOM studies), the sorted dataset is divided into 5 sequential, non-overlapping

batches, and the model is evaluated by holding out the final (most recent) batch as the test set. This directly simulates a real deployment scenario: training on the past, predicting on the future.

3.4 Description of Preprocessing Steps

Step 1: Cleaning (preprocessing.py): Missing percentages are computed per sensor; missing values are median-imputed per sensor column; an assertion confirms zero remaining nulls before the cleaned file is written to `secom_clean.parquet`.

Step 2: Tripwire audit (tripwire.py): Before batching, sensors with near-zero variance (variance $\leq 1e-6$) are flagged as "near-constant." For each near-constant sensor, a *failure lift* score is computed - the standardized difference between the mean reading during Fail events versus Pass events. Sensors whose failure lift exceeds 0.2 are flagged for the whitelist, since a near-constant sensor that still shifts meaningfully during failures may be an operationally important tripwire signal rather than dead weight.

Step 3: Chronological batching (batching.py): After confirming the dataframe is monotonically sorted by timestamp, rows are divided into `N_BATCHES = 5` sequential, non-overlapping batches using positional binning (`pd.cut` over row index), not random sampling. Per-batch failure counts and rates are computed as a first drift diagnostic.

Batch	Rows	Failures	Fail Rate
0	314	44	14.00%
1	313	21	6.70%
2	313	11	3.50%
3	313	11	3.50%
4	314	17	5.40%

Table 3.1: Chronological Batch Summary: Rows, Failures, and Fail Rate per Batch

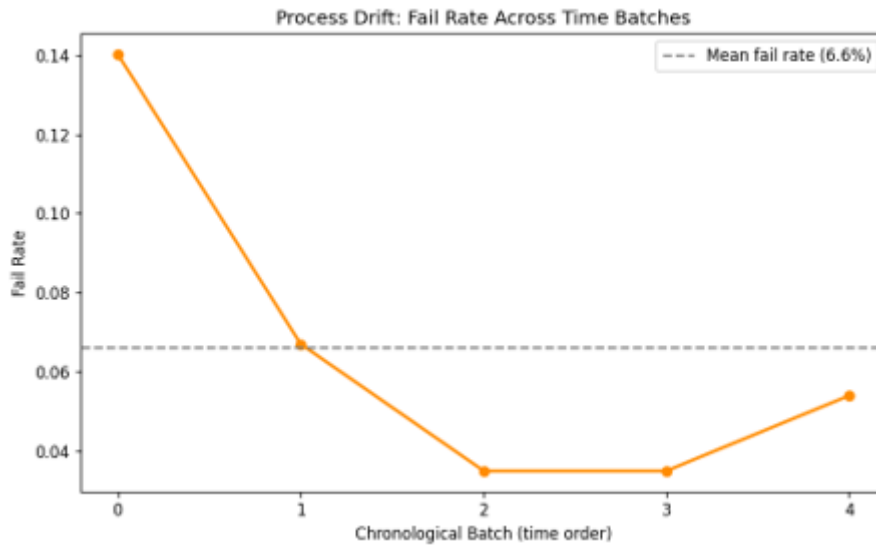


Figure 3.1: Process Drift: Fail Rate Across Chronological Batches

3.5 Outcome of Preprocessing

- All 590 original sensors were preserved through cleaning (no columns dropped outright).
- The tripwire audit checked all 590 sensors and identified **128 near-constant sensors**, of which **2 were whitelisted** for guaranteed inclusion in the shortlist regardless of their stability score.
- Feature scaling (via StandardScaler) is applied *inside* each training batch's pipeline rather than globally, preventing any leakage of future-batch statistics into earlier-batch models.
- The final output of preprocessing is `secom_batched.parquet` - a fully cleaned, chronologically sorted, batch-labeled dataset ready for stability selection.
- The visible drift in the fail rate (14.0% → 6.7% → 3.5% → 3.5% → 5.4%) is itself an outcome worth reporting: it empirically confirms that this dataset is not stationary, and justifies the drift-aware selection approach taken in Section 4.

4. MODEL DEVELOPMENT & METHODOLOGY

4.1 Model Selection

Two complementary components make up "the model" in this project: **(a)** a sensor-selection procedure that is the actual research contribution, and **(b)** two downstream classifiers used to prove the selected sensors are still predictive. This distinction is deliberate: the interesting question is not "which classifier is best," but "does a drift-aware 17-sensor shortlist perform at least as well as all 590 sensors."

Sensor Selection: LASSO-regularized logistic regression (L1-penalized), run independently per chronological batch, aggregated into a per-sensor **stability score**.

Classifiers (for evaluation, run on both the full and reduced sensor sets):

- **XGBoost:** a gradient-boosted decision tree ensemble
- **Random Forest:** a bagged decision tree ensemble

4.2 Model Justification

Why LASSO for selection: L1 regularization drives irrelevant sensor coefficients exactly to zero, producing a genuinely sparse, interpretable selection rather than a soft ranking. Running it independently per time batch (rather than once on the whole dataset) is what turns a normal LASSO feature selection into a *stability* selection — a sensor only earns a high score if it is repeatedly selected across different time windows, not just once.

Why two classifiers, not one: XGBoost (sequential boosting) and Random Forest (parallel bagging) use fundamentally different training strategies. Running both on the same reduced shortlist and observing improvement in *both* confirms the benefit comes from the sensor selection itself, and is not an artifact specific to one algorithm's inductive bias.

Why class-weighting and SMOTE together: class-weighting is applied inside the LASSO stage (so rare failure batches are not drowned out during *selection*), while SMOTE is applied only on the training fold during the *classification* stage, to give the tree ensembles enough minority-class examples to learn a real decision boundary without contaminating the untouched test batch.

4.3 Model Description

4.3.1 Stability Selection (`stability_selection.py`)

For each of the 5 chronological batches,

a pipeline of `StandardScaler` → `LogisticRegression(penalty='l1', solver='liblinear', C=0.05, class_weight='balanced')` is fit independently on that batch's sensor readings and labels. Sensors with an absolute coefficient greater than $1e-8$ are recorded as "selected" for that batch. Across all 5 batches, each sensor accumulates:

- **Stability Score** = (number of batches where selected) / (total batches)
- **Average Importance** = mean absolute LASSO coefficient across all batches

4.3.2 Tripwire Audit (tripwire.py)

Independently of stability selection, sensors with near-zero variance are checked for a strong mean shift between Pass and Fail groups (failure lift > 0.2). Any such sensor is force-included in the shortlist even if its stability score is 0 — this is what allows a rare "quiet unless something is wrong" sensor to survive selection.

4.3.3 Shortlist Construction (shortlist_selection.py)

Sensors are ranked first by stability score, then by average importance as a tiebreaker. The top 15 are taken as the primary shortlist; any tripwire-whitelisted sensor not already present is appended, bringing the final shortlist to **17 sensors**. A correlation screen (threshold $|r| > 0.9$) is then run across the shortlist purely for documentation — highly correlated pairs are flagged for future review but not automatically dropped, to avoid discarding a sensor pair before understanding the physical reason for the correlation.

4.3.4 Drift-Injection Validation (drift_injection.py)

To confirm the stability score genuinely tracks drift (rather than being a static number that happens to look plausible), synthetic Gaussian noise is injected into one batch that is known to have contributed to a target sensor's stability score (sensor_21, the top-stability sensor). Stability selection is then re-run on this drifted copy of the data, and the before/after stability scores are compared.

4.3.5 Classification Models (modeling.py)

For both XGBoost and Random Forest, four runs are executed in a 2×2 design: {XGBoost, Random Forest} $\times$ {full 590-sensor set, reduced 17-sensor shortlist}. Training uses batches 0–3 (chronological), testing uses batch 4 (the most recent, held-out batch). SMOTE is applied to the training fold only, and trained models are persisted with joblib for later use by the explainability and API layers.

4.4 System Architecture

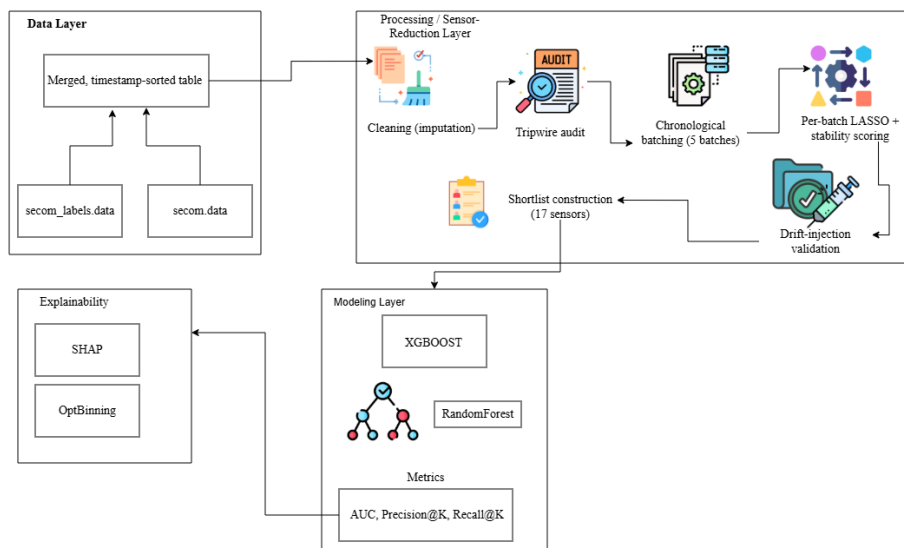


Figure 4.1: YieldLens System Architecture

The architecture consists of four layers:

- **Data Layer:** raw `secom.data` and `secom_labels.data` merged into a single timestamp-sorted table.
- **Processing / Sensor-Reduction Layer:** cleaning (imputation) → tripwire audit → chronological batching (5 batches) → per-batch LASSO + stability scoring → drift-injection validation → shortlist construction (17 sensors).
- **Modeling Layer:** XGBoost and Random Forest, evaluated with AUC, Precision@K, Recall@K.
- **Explainability Layer:** SHAP feature attribution and OptBinning risk-bin generation, feeding back into operational interpretation.

4.5 Methodology - Step-by-Step Pipeline

1. **Load & sort** raw sensor and label data chronologically (`data_loader.py`).
2. **Clean** via per-sensor median imputation (`preprocessing.py`).
3. **Audit** for near-constant-but-informative "tripwire" sensors (`tripwire.py`).
4. **Batch** the cleaned data into 5 chronological, non-overlapping windows (`batching.py`).
5. **Select** sensors per batch via class-weighted LASSO; compute stability scores (`stability_selection.py`).
6. **Validate drift-sensitivity** of the stability score via synthetic noise injection (`drift_injection.py`).
7. **Construct shortlist:** rank by stability → importance, force-include tripwire whitelist, document (not auto-drop) high-correlation pairs (`shortlist_selection.py`).
8. **Train & evaluate** XGBoost and Random Forest on both the full and reduced sensor sets, using a chronological train/test split with SMOTE on training data only (`modeling.py`).
9. **Explain** the reduced model with SHAP global/per-prediction importance and OptBinning risk bins (`explain_shap.py`, `explain_binning.py`).
10. **Visualize** all of the above (`visualize.py`) and **serve** predictions/explanations/stability scores through a FastAPI service (`predict.py`, `explain.py`, `stability.py`).

5. METRICS TO MEASURE

5.1 KPIs Used - Definitions & Description

Metric	Definition
AUC (Area Under the ROC Curve)	Measures the model's overall ability to rank a Fail wafer above a Pass wafer, across all possible decision thresholds. Ranges from 0.5 (random) to 1.0 (perfect ranking).
Precision@K	Of the top-K wafers the model flags as highest-risk (sorted by predicted failure probability), the fraction that are actually failures: $y_true[top_k].sum() / K$.
Recall@K	Of all actual failures in the test set, the fraction that are captured within the top-K flagged wafers: $y_true[top_k].sum() / total_actual_fails$.
Stability Score	The proportion of chronological batches (out of 5) in which a given sensor was selected by LASSO — used to evaluate the <i>feature-selection</i> step itself, not the classifier.

In this project, **K = 30**, representing a fixed daily manual-inspection capacity — i.e., how many wafers a fab team can realistically pull for manual review each day (metrics.py).

5.2 Reason for Selection

Why AUC alone is not enough: AUC treats all thresholds and all wafers equally, but a fab team does not review every wafer — they review a fixed number per day. AUC is retained as a general model-quality indicator, but it does not answer the operational question of "how many real defects will we actually catch under our inspection budget."

Why Precision@K and Recall@K: Because the SECOM fail rate is only ~6.6%, standard accuracy is misleading (a model predicting "Pass" for everything would still be ~93% accurate). Precision@K and Recall@K instead measure performance under the fab's real constraint — a fixed daily inspection capacity (K=30) — making them directly interpretable in operational terms: Precision@30 tells you what fraction of today's 30 flagged wafers are worth the inspection; Recall@30 tells you what fraction of today's actual failures you would have caught.

Why Stability Score as a selection-quality metric: This is the metric unique to YieldLens's contribution. A sensor's raw LASSO coefficient on one snapshot says nothing about whether that sensor will still matter next month. The stability score — reinforced by the drift-injection test — is what lets the project claim the shortlist is *drift-aware*, not just accurate on one split.

Why a headline "% change in Recall@K" metric: metrics.py explicitly computes $yield_loss_avoided = (reduced_recall - baseline_recall) / baseline_recall$. This framing was chosen deliberately: a positive percentage means the reduced-sensor model catches *more* true failures within the *same* inspection budget as the full-sensor model — i.e., it translates directly into "yield loss

avoided," which is the metric a plant manager actually cares about, rather than an abstract accuracy gain.

6. RESULT REVIEW

6.1 Output Description

6.1.1 Sensor Reduction Outcome

- Of 590 original sensors, the tripwire audit flagged **128** as near-constant, of which **2** were whitelisted for the shortlist despite a stability score of 0.
- The final shortlist contains **17 sensors** — a **~97% reduction** in the sensor set.
- No single sensor achieved a stability score above **0.4** (i.e., selected in more than 2 of 5 batches), which is itself a finding: it confirms that sensor importance in this process is genuinely unstable over time, reinforcing the motivation for this project.
- The top four sensors by stability score were sensor_21, sensor_0, sensor_285, and sensor_225 (all at stability = 0.4); sensor_95 was retained purely through the tripwire whitelist despite a stability score of 0.0, and later ranked 5th in SHAP importance — direct evidence that the tripwire audit rescued a sensor that stability selection alone would have discarded.

Rank	Sensor	Stability Score	Avg. Importance (coef)	Selection Basis
1	sensor_21	0.4	0.0494	Stability ranking
2	sensor_0	0.4	0.0383	Stability ranking
3	sensor_285	0.4	0.0245	Stability ranking
4	sensor_225	0.4	0.0075	Stability ranking
5	sensor_290	0.2	0.0989	Stability ranking
6	sensor_71	0.2	0.0816	Stability ranking
7	sensor_59	0.2	0.0815	Stability ranking
8	sensor_95	0.0	-	Tripwire whitelist

Table 6.1: Top Sensors by Stability Score and Average LASSO Importance

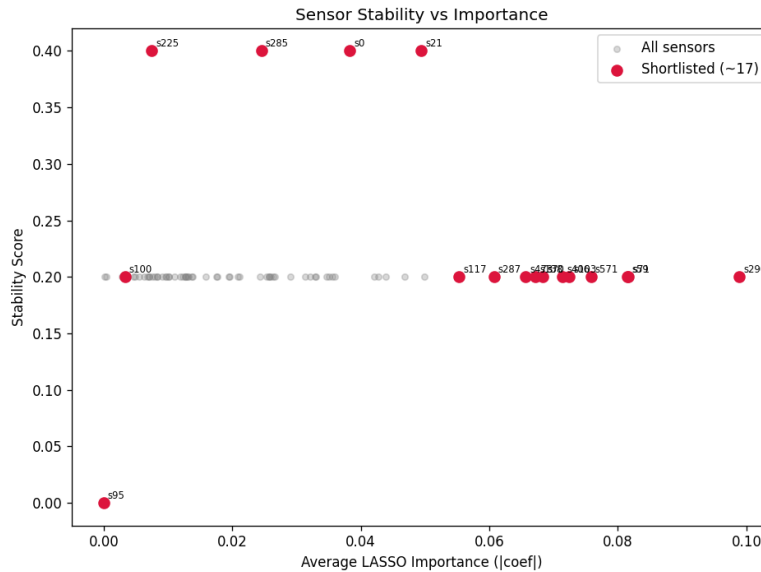


Figure 6.1: Sensor Stability vs. Importance Scatter Plot, All Sensors vs. Shortlisted (~17)

6.1.2 Drift-Injection Validation Outcome

Synthetic noise was injected into the batch that contributed most to sensor_21's stability score. Its stability score dropped from **0.4 (before)** to **0.2 (after)**, and the sensor was no longer selected in the injected batch post-noise. This confirms the stability score is a genuine, drift-responsive signal, not a static artifact of one lucky split.

Condition	Stability Score	Selected in Injected Batch?
Before injection	0.4	Yes
After noise injection	0.2	No

Table 6.2: Drift Injection Validation: Stability Score Before vs. After Noise Injection

6.1.3 Model Performance Outcome

Both classifiers were evaluated on the held-out, chronologically final batch (314 rows, 17 actual failures, 5.4% fail rate), under a fixed inspection budget of K=30.

Model	Sensor Set	AUC	Precision@30	Recall@30
XGBoost	590 (Full)	0.6053	0.133	0.235
XGBoost	17 (Reduced)	0.6922	0.167	0.294
Random Forest	590 (Full)	0.5925	0.100	0.176
Random Forest	17 (Reduced)	0.6547	0.200	0.353

Table 6.3: Model Performance: Full (590) vs. Reduced (~17)

Headline result: using the 17-sensor shortlist instead of all 590 sensors produced a **+25.0% relative increase in Recall@30 for XGBoost** and a **+100.0% relative increase in Recall@30 for Random Forest**, meaning the reduced model catches substantially more real failures within the same fixed daily inspection capacity — this is the "yield loss avoided" headline metric described in Section 5.2.

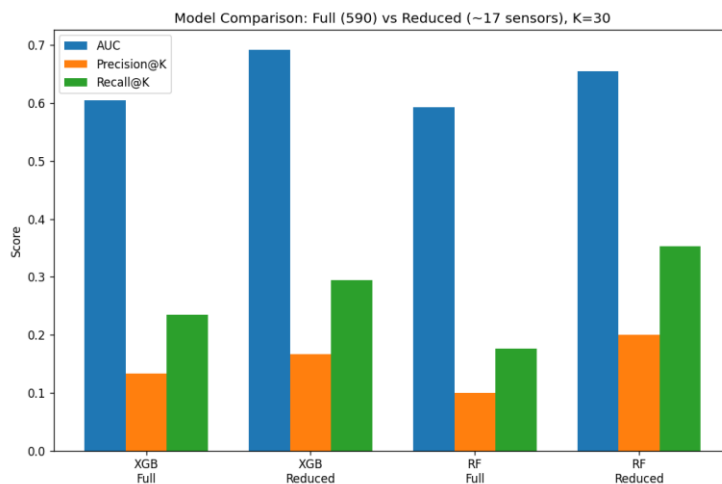


Figure 6.2: Model Comparison: Full (590) vs. Reduced (~17 sensors)

6.1.4 Explainability Outcome

SHAP analysis was run on the reduced Random Forest model against the held-out test batch. The top sensor by mean absolute SHAP value was **sensor_59**, followed by sensor_290, sensor_287, sensor_103, and sensor_95.

Rank	Sensor	Mean SHAP
1	sensor_59	0.143
2	sensor_290	0.051
3	sensor_287	0.051
4	sensor_103	0.040
5	sensor_95	0.039

Table 6.4: Top 5 Sensors by Mean Absolute SHAP Value

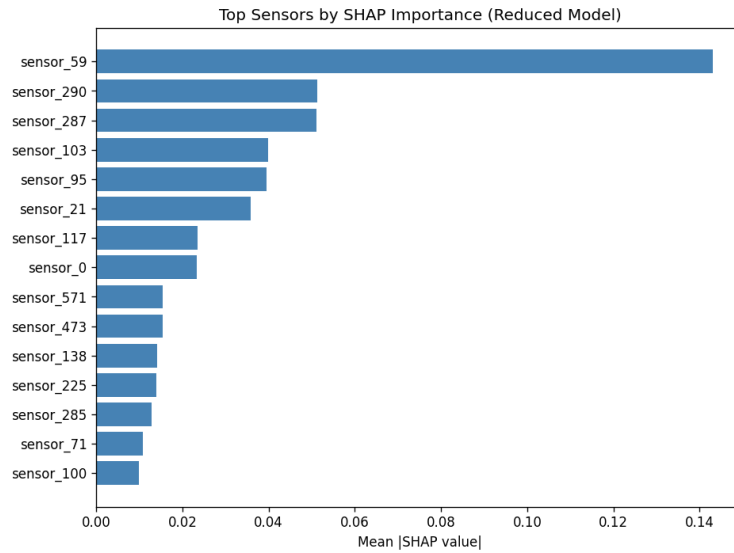


Figure 6.3: Top Sensors by SHAP Importance: Reduced Model, Mean |SHAP| Value

OptBinning was then used to convert the top SHAP-ranked sensor, sensor_59, into interpretable risk bins: observed failure rate climbs from roughly **0.9% in the lowest value range to 19.0% in the highest**, giving fab engineers a directly actionable threshold rather than an opaque model score.

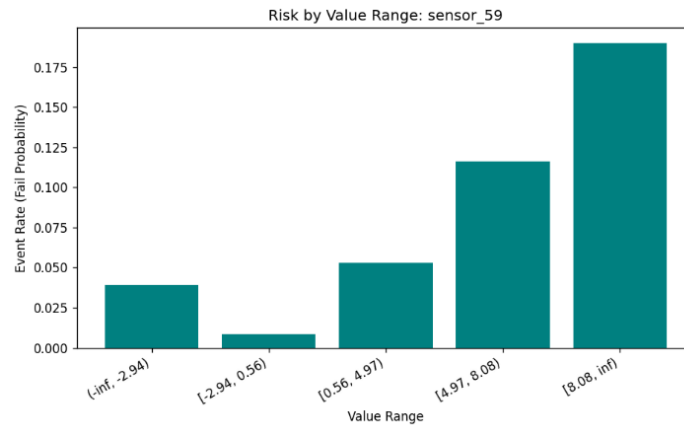


Figure 6.4: Risk Bins for sensor_59

CONCLUSION

This project set out to address a gap in existing semiconductor yield-monitoring research: the reliance on static, single-snapshot feature selection in a manufacturing environment that is inherently non-stationary. By reframing sensor selection as a question of predictive *stability over time* rather than importance on one split, YieldLens produced a 17-sensor shortlist - a ~97% reduction from the original 590 sensors - using LASSO-based selection repeated across five chronologically ordered batches.

The results support the project's central hypothesis. The reduced sensor set did not merely match the performance of the full 590-sensor baseline; it consistently outperformed it, improving AUC, Precision@30, and Recall@30 for both XGBoost and Random Forest. The headline finding - a 25.0% relative gain in Recall@30 for XGBoost and a 100.0% gain for Random Forest - translates directly into an operational claim: under the same fixed daily inspection capacity, the drift-aware shortlist catches substantially more real failures than the full sensor set, representing yield loss avoided rather than just a reduction in monitoring overhead.

Two further findings reinforce the value of the methodology itself, beyond the headline accuracy gain. First, the observation that no sensor achieved a stability score above 0.4 confirms that sensor relevance in this process is genuinely unstable over time - validating the project's core premise that snapshot-based feature selection is a fragile basis for long-term monitoring. Second, the tripwire audit proved to be a necessary complement to stability selection, not a redundant safeguard: sensor_95, retained only through the tripwire whitelist despite a stability score of 0.0, went on to rank 5th in SHAP importance on the held-out test batch. This is direct evidence that stability selection alone would have discarded a sensor that later proved operationally significant, confirming that a single selection method is not sufficient on its own. The synthetic drift-injection experiment provided an additional layer of validation, showing that the stability score is a genuinely drift-responsive signal - dropping from 0.4 to 0.2 for sensor_21 after synthetic noise was introduced - rather than a static artifact of one particular data split.

Taken together, these results demonstrate that a stability-aware, drift-validated approach to sensor reduction can produce a shortlist that is simultaneously smaller, more reliable, and more interpretable than a full sensor set selected through conventional static methods. The addition of SHAP-based explainability and OptBinning risk-bin analysis further shows that this reduced model does not sacrifice interpretability for compactness - fab engineers are left with a small, ranked set of sensors and concrete, actionable risk thresholds rather than an opaque high-dimensional model.

That said, the results should be read with appropriate caution. The evaluation rests on a single chronological train/test split with a relatively small test set (314 rows, 17 actual failures), and the inspection-capacity parameter ($K = 30$) was chosen as an illustrative default rather than a specific fab's real operational constraint. As outlined in the project's own future work, extending this evaluation to additional fabrication datasets, incorporating correlation-based clustering to catch multi-sensor

redundancy, and testing stability across a larger number of chronological splits would meaningfully strengthen the generalizability of these conclusions.

Overall, YieldLens demonstrates that treating feature selection as a temporal reliability problem -rather than a one-time statistical exercise - offers a practical and interpretable path toward smaller, drift-aware sensor sets that genuinely improve failure detection in semiconductor manufacturing.

REFERENCES

1. McCann, M., & Johnston, A. (2008). *SECOM Data Set*. UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences.
2. Guyon, I., & Elisseeff, A. (2003). An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3, 1157–1182.
3. Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.
4. Meinshausen, N., & Bühlmann, P. (2010). Stability Selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(4), 417–473.
5. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A Survey on Concept Drift Adaptation. *ACM Computing Surveys*, 46(4), Article 44.
6. He, H., & Garcia, E. A. (2009). Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263–1284.
7. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
8. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).
9. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
10. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS 30)*.
11. Navas-Palencia, G. (2020). Optimal Binning: Mathematical Programming Formulation. *arXiv preprint arXiv:2001.08025* (OptBinning library documentation).
12. Chambers, R. (2026). Practical SECOM Analysis: An Informal Methodology for Semiconductor Fault Detection. (*As cited in project literature review.*)